

TwinForge — Project Profile

Track: Track 1 — Development of Multimodal Content Creation Tools

Team name: TwinForge (solo — Emmanuel Duke)

Application name: TwinForge

Submission repository (app tier): <https://github.com/dukemawex/twinforge>

Deadline: August 6, 2026 (UTC+8 23:59)

1. Project Background

Professional talking-head/spokesperson video is expensive and slow: studios, actors, reshoots.

Cloud "AI avatar" tools exist but are closed, per-seat expensive, and send user likeness to

third-party servers. TwinForge is an **open, self-hostable digital-human video studio** whose

core generation runs **locally on an AMD Radeon GPU via the ROCm stack** — no reliance on

closed online APIs for the core creation task. A creator uploads a short reference clip and a

voice sample, and TwinForge produces scripted, multilingual talking-human videos on demand.

2. Target Users & Application Scenarios

- **New-media creators & educators** — turn a script into a presenter video without filming.
- **SMB / commercial visual design** — product explainers, localized ad reads, onboarding.
- **Short-video production** — batch-generate variations from a CSV of scripts.
- **EdTech** (author's domain, MyRabbai) — auto-generate lesson presenters in local languages.

3. System Architecture (Hybrid, two tiers)

App tier (already built — dukemawex/twinforge , Next.js + TypeScript + Tailwind + shadcn/ui):

frontend studio, auth, Postgres (users/twins/jobs/video metadata), blob storage with signed

uploads, a render-job queue, a GPU proxy route, and a webhook (/api/webhooks/gpu) the GPU

server calls on job completion. The app tier performs **no model math** — it only orchestrates.

GPU inference tier (this submission's ROCm component — runs on AMD Radeon cloud GPU):

a PyTorch + ROCm FastAPI service exposing the contract the app already expects:

- POST /twins/train — build a twin from reference media + voice sample
- GET /twins/:jobId/status
- POST /render — script → talking-human video (face animation + voice + lip-sync)
- POST /render/batch — CSV-driven batch generation
- GET /system/gpu-status — live gpuModel / rocmVersion / vram / utilization

Data flow: **input** (reference media, voice, script) → **processing** (ROCm inference on

Radeon GPU) → **output** (rendered MP4 in blob storage, surfaced in the Library). This is the

complete input–processing–output workflow required by the rubric.

4. Model & Algorithm Introduction

Core pipeline uses **open-source models**, permitted for engineering modification:

- **Voice cloning / TTS** — open TTS (e.g. XTTS-family / Coqui-style) for multilingual speech.
- **Face animation & lip-sync** — open talking-head models (e.g. SadTalker / Wav2Lip-style)

driven by the cloned audio to animate the reference identity.

- **Diffusers** for background/stylization and frame enhancement.

- LoRA / lightweight & distilled variants permitted for speed on a single Radeon GPU.

No closed-source online API is used for the core generation. At least one key inference process (face animation + lip-sync render) runs locally on the AMD Radeon GPU.

5. Adaptation for AMD Radeon GPU / ROCm

- Runs in a **ROCm PyTorch** container on Radeon Cloud (per the Radeon Cloud User Guide).
- Deployed either as a JupyterLab/SSH instance running the FastAPI inference server, or via a dedicated vLLM endpoint for the script-assist LLM step (OpenAI-compatible, Radeon-served).
- Optimizations for single-GPU throughput: fp16 inference, model offload between stages, batched frame rendering, and VRAM-aware scheduling reported live via `/system/gpu-status`.
- Free Radeon-served model APIs (Qwen / DeepSeek) power the in-app **script assistant**, while the **core video generation stays on the local Radeon GPU** — satisfying the core-inference rule.

6. Deliverable Form

Web UI (TwinForge studio: Landing, Twin Studio wizard, Create Video, Batch Studio, Library, Settings) backed by the ROCm inference server.

7. Status & Remaining Work Before Final Submission

- [x] App tier built (`dukemawex/twinforge`).
- [x] Fork + submission PR (this).
- [] Radeon Cloud GPU access provisioned (in progress).
- [] ROCm inference server implemented & run on Radeon GPU (the scored component).
- [] Demo video (3–5 min) showing real execution on AMD Radeon GPU.
- [] Project Profile exported to PDF + PPT/poster.